

PROPUESTA DE ARQUITECTURA PARA REQUERIMIENTO DE DEVSU

Nombre: Christian Fabara

ANÁLISIS DEL REQUERIMIENTO

- 1) El cliente solicita la construcción de un sistema de banca en línea, a través del cual puede realizar varias funcionalidades como movimientos, transferencias y pagos.
- 2) Dicho sistema va a interactuar con 2 sistemas ya existentes que proporcionan las siguientes funcionalidades:
 - a. Plataforma Core que contiene la información básica del cliente, como movimientos, productos, etc.
 - b. Sistema de información complementaria

SOLUCION PROPUESTA

De acuerdo a lo solicitado por el cliente, el sistema deberá tener las siguientes características:

BACKEND:

El backend deberá estar desarrollado en lenguaje JAVA, debido a que es un lenguaje robusto y multiplataforma, lo que le permite implementarse en cualquier entorno o sistema operativo.

1) FRAMEWORKS UTILIZADOS:

1. SpringBoot: Marco de trabajo para generar aplicaciones orientadas a microservicios. Debido a su estructura y características, permite la construcción de Apis independientes de manera fácil. Los Apis deben ser de tipo REST, debido a que permiten su manejo de manera desacoplada e independiente de estados, lo que facilita su uso en microservicios.
2. Hibernate JPA: Junto con SpringBoot, se puede utilizar en el sistema Hibernate, que es un framework para el manejo de la capa de persistencia o acceso a Datos. JPA a su vez va a permitir definir las entidades, relaciones y queries que se van a implementar a través de Hibernate. Su principal función es garantizar a todo el sistema el acceso a datos a través de objetos persistentes.

2) MÓDULOS DEL SISTEMA:

- **Api Data:** En este módulo estará toda la funcionalidad de acceso y manejo de base de datos. La base de datos que se debe manejar es de tipo SQL, y podría ser Oracle o PostgreSQL.
- **Api Audit:** Para el módulo de auditoría se propone hacer uso de Hibernate Envers, que aprovecha la implementación de Hibernate, para poder almacenar en base de datos pistas de auditoría sobre las entidades desarrolladas.
- **Api Security:** Contiene las Apis necesarias para brindar autenticación y autorización al usuario final, y su interacción con otros sistemas de manera óptima.
- **Api OnBoarding:** Contiene las Apis utilizadas en el proceso de registro y creación del usuario (OnBoarding)
- **Api Transaccional:** Contiene las Apis de las funcionalidades del sistema, como pagos, transferencias, consultas e históricos.
- **Api Integration:** Contiene las Apis de integración con los 2 sistemas ya existentes, como son el Core y el de información básica.

- **API Notifications:** Aquí se encontrará la funcionalidad tanto de envío de correos como la generación de mensajes.

Notificaciones por email: Para el envío de notificaciones por correo, se utilizarán las librerías propias de java.

Notificaciones por SMS y/o WhatsApp: Para el manejo de notificaciones de tipo SMS o de WhatsApp se puede hacer uso de Twilio, que es un proyecto externo que expone APIS para el envío de este tipo de mensajes.

Dichos módulos pueden ser desplegados individualmente en contenedores EC2, con el fin de desacoplar cada funcionalidad.

3) SEGURIDADES

Para el manejo de seguridades a nivel de backend, al hacer uso de Apis tipo REST, SpringBoot permite la fácil adaptación a manejo de mecanismos de seguridad como JWT y Oauth2. Para ello, el backend deberá implementar Spring Security, garantizando el manejo del servidor externo de autenticación que tiene la empresa.

En cuanto al flujo a utilizarse para el manejo de seguridad, se recomienda el uso de openId Connect junto con Oauth2, ya que permite potenciar la seguridad añadiendo una capa de seguridad sobre Oauth2.

Bajo esta premisa, el flujo que se podría utilizar es **Authorization code Flow** ya que permite intercambiar un simple código de autorización, por un token de un solo uso.

FRONT END

En cuanto al front end, este debe estar desacoplado al backend para lo cual se propone lo siguiente:

Entorno visual de tipo SPA: debido a que se requiere un entorno visual de tipo SPA, se puede utilizar Angular o React para poder proporcionar dicho front end.

Algo que se debe considerar con cualquiera de las 2 tecnologías es que dicho front end debe estar lo suficientemente dividido en componentes de tal manera que se pueda modificar un componente sin afectar a otros.

Entorno visual Móvil: Para la aplicación móvil se propone estas alternativas:

- **IONIC:** Debido a que utiliza HTML, esta tecnología permite generar aplicaciones móviles independientemente de su plataforma (Android o IOS).
- **React Native:** Permite la generación de aplicaciones móviles híbridas, pero de tipo JavaScript.

MECANISMOS DE SEGURIDAD BIOMÉTRICA

Para poder implementar el desbloqueo facial en el entorno visual móvil, se deberá hacer uso de FaceIO, que proporciona la funcionalidad de desbloqueo facial con React Js.

NORMATIVA

Debido a que el sistema está orientado a la banca y su uso es a través de la web, la seguridad y confiabilidad son importantes. Por tal razón el sistema debe estar construido basado en buenas prácticas o estándares que actualmente son importantes. Uno de ellos es OWASP Top 10, el cual define los riesgos más críticos que actualmente están vigentes para poder ayudar a implementar mecanismos de protección sobre los sistemas.

Por otra parte, para garantizar la seguridad, es importante que el sistema también tome en cuenta las mejores prácticas propuestas en la ISO: 27001, que es un estándar para garantizar la seguridad de la información.

Y para proteger el manejo de transacciones y pagos electrónicos, el aplicativo debe estar basado en las mejores prácticas propuestas en el estándar PCI DSS, con el fin de generar confianza en el uso de medios electrónicos y reducir los riesgos de pérdida de datos y fraudes.

INFRAESTRUCTURA

Para poder garantizar el correcto funcionamiento del sistema en cuanto a alta disponibilidad, tolerancia a fallos, seguridad, y monitoreo, se remienda el uso de infraestructuras orientadas a nube, como AWS, que proporcionan diferentes herramientas para estos propósitos. Para este sistema se recomendaría lo siguiente:

AMAZON EC2: El sistema debe estar desplegado en Amazon EC2 tanto en back y front, ya que proporciona máquinas virtuales en la nube, orientadas a contenerización con Docker.

Al usar EC2 se facilita la asignación de recursos como memoria, procesamiento, almacenamiento, y por sobre todo, contenerización, por lo que se pueden generar máquinas virtuales a demanda, y también su escalado automático.

AMAZON EKS: Si se desea utilizar Kubernetes, El sistema debe estar desplegado en Amazon EKS ya que tiene las mismas funcionalidades de EC2, pero orientado a Kubernetes.

ELASTIC LOAD BALANCE: Esta característica de AWS ofrece el balanceo de carga para varios componentes de sistema, como por ejemplo aplicaciones, red y puerta de enlace.

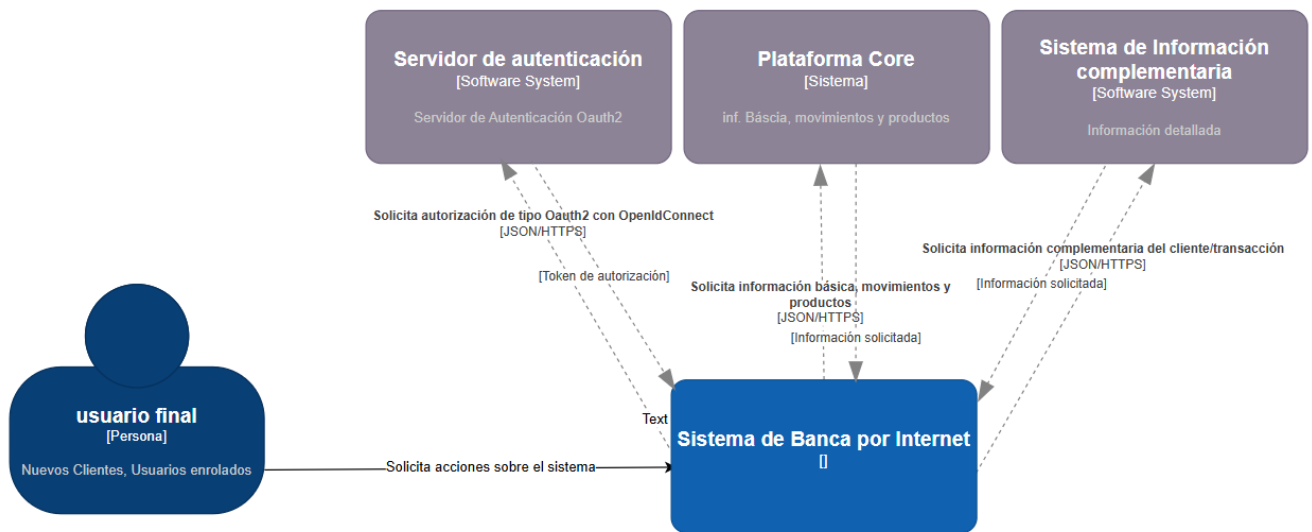
En el caso del balanceador de carga sobre aplicaciones, permite garantizar, la disponibilidad del servicio distribuyendo su carga entre varias instancias del sistema, que se pueden configurar con EC2.

S3: Permite el almacenamiento de datos seguro en la nube. Si en algunas de las APIs, se requiere el manejo de cualquier tipo de documento, S3 permite su almacenamiento, y la correcta distribución en carpetas.

Por último, para el monitoreo, AWS permite supervisar cada una de las partes de la infraestructura del sistema, a nivel de instancia, aplicación o servidor.

DIAGRAMA DE CONTEXTO

De acuerdo a la solución planteada el diagrama de concepto quedaría de la siguiente forma:



Como se muestra en la imagen, el nuevo sistema recibirá todas las peticiones provenientes del usuario, y las atenderá a través de llamadas a APIs, devolviendo el resultado de transacciones o acciones solicitadas a través de la aplicación web o móvil.

DIAGRAMA DE CONTENEDORES

En el diagrama de contenedores, se puede visualizar el manejo del balanceador de carga para dos bajo la infraestructura de AWS, con el fin de brindar alta disponibilidad al sistema propuesto.

Cada instancia va a tener contenedores de tipo EC2, uno para el front end web, uno para el app móvil y para las APIs del Backend, el cual interactúa con los sistemas externos a través de APIs Restful. Las APIs van a permitir realizar las siguientes funciones, desde su propio módulo:

- Autenticación y autorización con el servidor de OAuth2.
- Consulta de información con el sistema Core
- Consulta de información del sistema de información complementaria
- Generación de transacciones bancarias
- Notificaciones vía email y SMS o WhatsApp

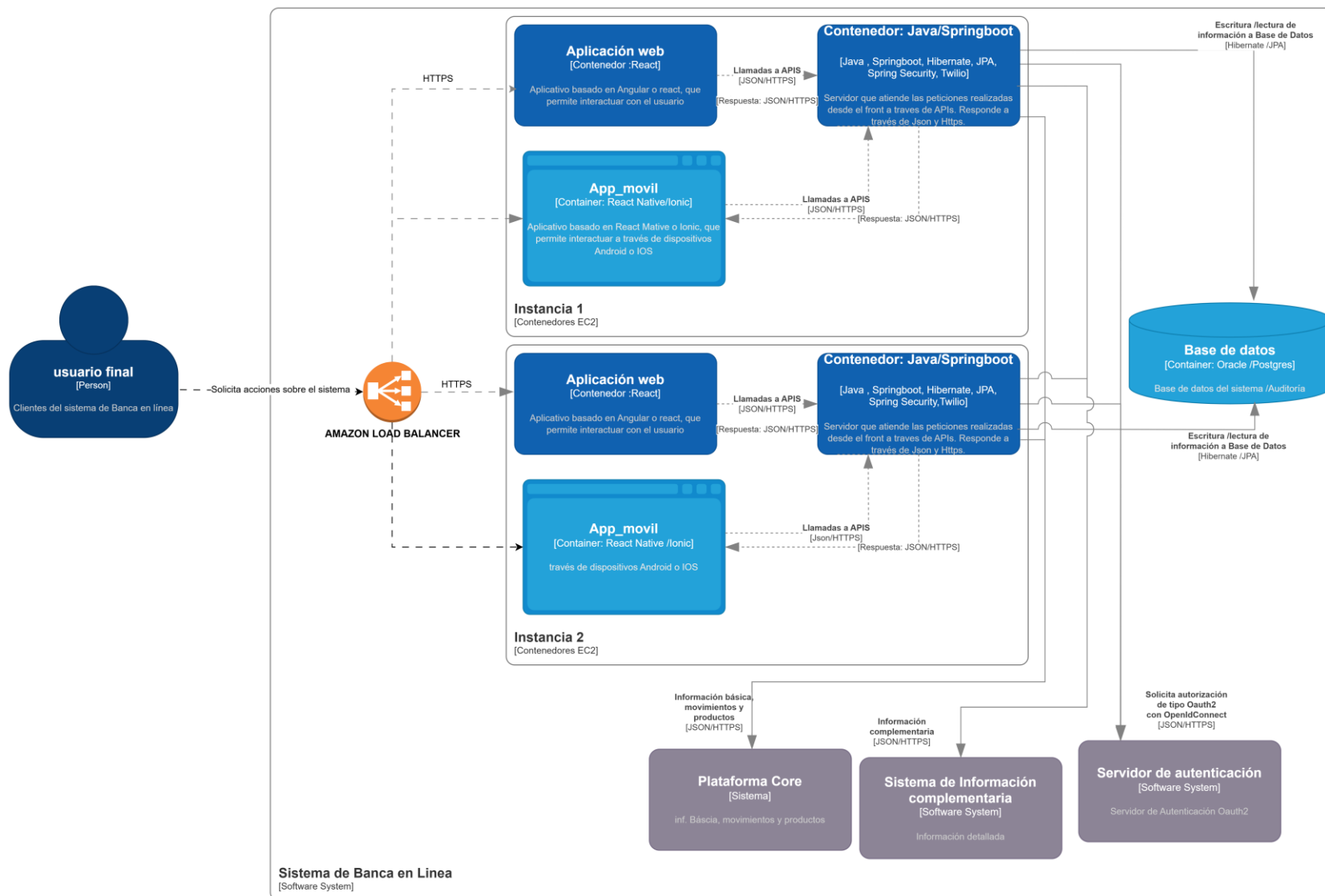
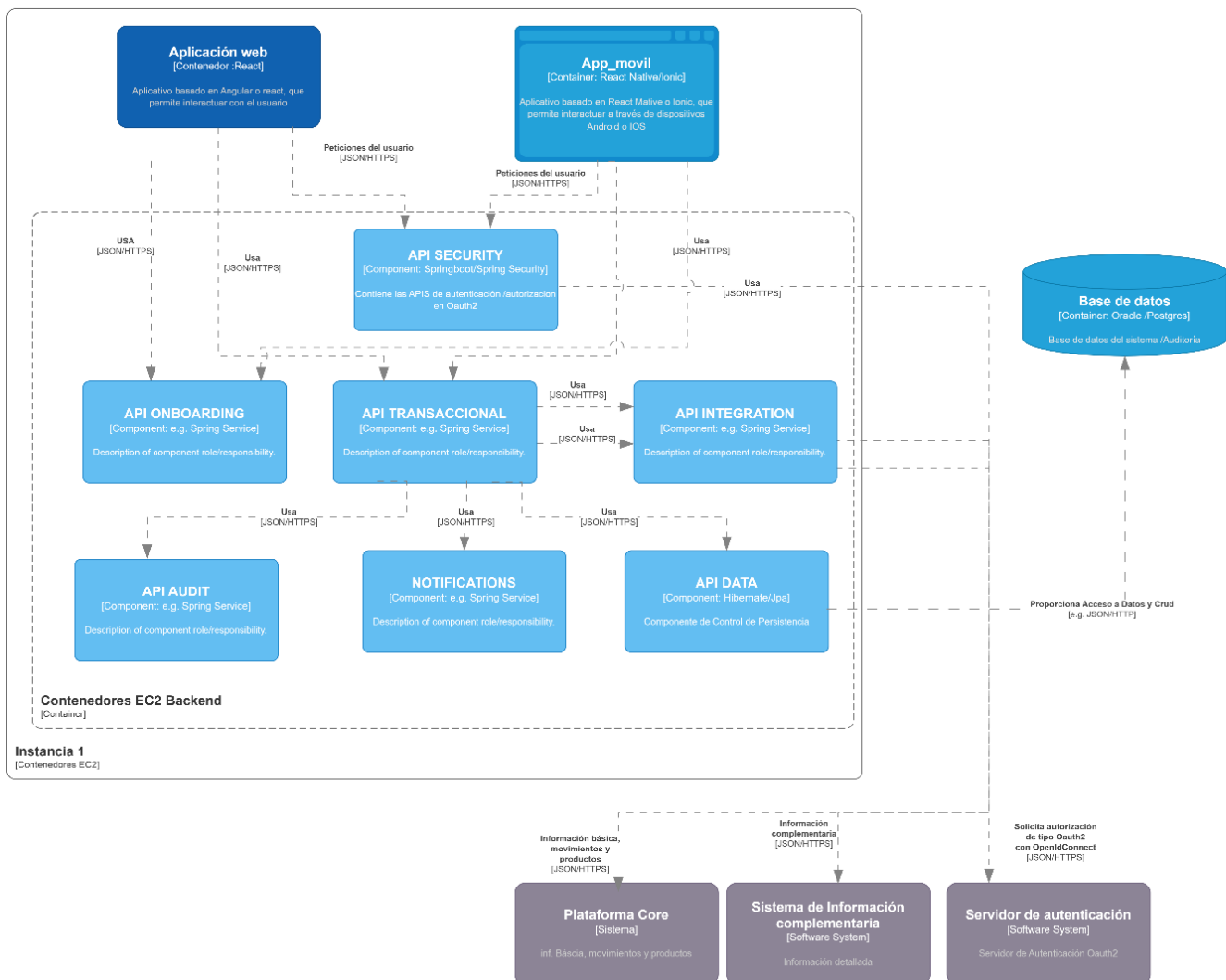


DIAGRAMA DE COMPONENTES



De acuerdo al diagrama de componentes, el Aplicativo funcionaría de la siguiente manera:

- Las peticiones se originan a través de cualquiera de los 2 entornos visuales.
- Las peticiones iniciales son para obtener el token de autorización o Bearer token desde las APIs de seguridad (API SECURITY).
- Con el token obtenido, las demás peticiones referentes a las transacciones del sistema se originarán en el frontend, apuntando al API TRANSACCIONAL, el cual puede hacer uso de las APIs de auditoría (API AUDIT), integrarse a otros sistemas (API INTEGRATION), enviar notificaciones referentes a la transacción (API NOTIFICATIONS) o de manejo de datos (API DATA), dependiendo de la lógica del negocio que se deba implementar.
- De igual manera, para el proceso de OnBoarding, las peticiones se originarán en el frontend y se invocarán a través de servicios Restful.
- Cada una de las APIs corresponde a un contenedor EC2, y se invocarán a través de peticiones tipo REST con HTTPS, sobre el cual se enviará el Bearer token generado al inicio de la sesión.
- Cada API podrá interactuar con otra a través de servicios, utilizando inyección de dependencia.
- La respuesta de cada transacción será auditada a través de las APIs de API AUDIT gracias a Spring Security, a nivel de Entidad.